

Machine Learning Notes

From ML fundamentals and preprocessing to algorithms, deep learning, evaluation and deployment.

Research-based study notes • Updated September 2026

1. Machine Learning Fundamentals

Machine learning learns a mapping from data to predictions or decisions. A dataset typically contains features X and a target y . Training estimates model parameters from a training set; evaluation estimates performance on unseen data.

Three major settings

- Supervised learning: labeled examples; regression and classification.
- Unsupervised learning: discover structure without target labels; clustering and dimensionality reduction.
- Reinforcement learning: learn actions from rewards through interaction.

2. End-to-End ML Workflow

Problem definition → data collection → data cleaning → exploratory analysis → train/validation/test split → preprocessing → model training → hyperparameter tuning → evaluation → deployment → monitoring.

The most common beginner mistake is jumping directly to model training. Good feature definitions, leakage prevention and evaluation design often matter more than choosing a sophisticated algorithm.

3. Data Preprocessing

Missing values

Use domain-aware imputation. Numerical values can use median/mean depending on the problem; categorical values can use the most frequent category or an explicit missing category.

Scaling

Standardization commonly transforms x to $(x - \text{mean}) / \text{standard deviation}$. Min-max scaling maps values to a chosen range. Tree-based models usually need less scaling than distance- or gradient-sensitive models.

Categorical variables

One-hot encoding creates binary columns for categories. Be careful with high-cardinality features and unknown categories at inference time.

4. Train / Validation / Test

Training data fits parameters. Validation data guides model/hyperparameter choices. The test set should be held out for final unbiased evaluation.

Data leakage

Leakage occurs when information unavailable at prediction time enters training or preprocessing. Examples include computing a normalization statistic using the full dataset before splitting, or using a future outcome as a feature.

5. Regression

Regression predicts a continuous value.

Linear regression

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Common metrics: MAE, MSE, RMSE and R^2 . MAE is easy to interpret in target units. RMSE penalizes larger errors more strongly.

6. Classification

Classification predicts a discrete class.

Logistic regression

Logistic regression models class probability using the sigmoid function: $\sigma(z) = 1/(1+e^{-z})$. A threshold converts probability into a class prediction.

Metrics

- Accuracy: correct predictions / total; can be misleading for imbalanced data.
- Precision: $TP/(TP+FP)$.
- Recall: $TP/(TP+FN)$.
- F1: harmonic mean of precision and recall.
- ROC-AUC: ranking quality across thresholds.
- PR-AUC: often informative for rare positive classes.

7. Core Algorithms

Decision trees

Recursive splits create a tree of rules. They handle nonlinear relationships and mixed feature scales but can overfit.

Random forests

An ensemble of randomized decision trees. Averaging/voting reduces variance and usually provides a strong baseline.

Gradient boosting

Builds models sequentially to reduce residual errors. Implementations include gradient-boosted decision trees and modern libraries such as XGBoost, LightGBM and CatBoost.

k-NN

Predicts using nearby training examples. Sensitive to feature scaling and can become expensive at inference for large datasets.

SVM

Finds a separating boundary with a margin; kernels can model nonlinear boundaries. Scaling is generally important.

8. Clustering & Dimensionality Reduction

K-means

Partitions points into k clusters by minimizing within-cluster squared distances to cluster centroids. You must choose k and scaling can affect results.

Hierarchical clustering

Builds a hierarchy of clusters, which can be visualized as a dendrogram.

PCA

Principal Component Analysis projects data into orthogonal directions of maximum variance. Standardization is often appropriate when features use different units.

9. Overfitting, Underfitting & Regularization

Underfitting means the model is too simple to capture the pattern. Overfitting means it learns training-specific noise and generalizes poorly.

Bias–variance

High bias is associated with overly simple models; high variance is associated with models that are too sensitive to training data.

Regularization

L2 adds a squared-weight penalty; L1 adds an absolute-weight penalty and can encourage sparse coefficients. Regularization strength is a hyperparameter.

10. Cross-Validation & Hyperparameter Tuning

K-fold cross-validation splits training data into k folds and repeatedly trains/evaluates across different held-out folds. It provides a more stable estimate than one split when data is limited.

Grid vs randomized search

Grid search evaluates specified combinations. Randomized search samples configurations and can explore large hyperparameter spaces more efficiently.

11. Neural Networks & Deep Learning

A neural network is composed of layers of parameterized transformations. A typical layer computes $z = Wx + b$ and an activation such as ReLU.

Training

Forward pass → loss calculation → backpropagation → optimizer update. Common optimizers include SGD and Adam. Learning rate, batch size and regularization strongly affect training.

Common architectures

- CNNs: especially useful for spatial/image patterns.
- RNN/LSTM/GRU: sequence modeling; many modern NLP systems use Transformer architectures instead.
- Transformers: attention-based architecture widely used for language, vision and multimodal models.

12. ML Engineering with scikit-learn

scikit-learn provides estimators, preprocessing, model selection, evaluation and pipelines.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])
```

```
] )
pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)
```

A Pipeline is useful because preprocessing is fitted within the training process and can be safely reused during prediction. This helps prevent leakage.

13. Model Deployment & Monitoring

Deployment turns a trained model into a service or embedded component. Save the model together with preprocessing and feature definitions, not just the final estimator.

Production concerns

- Input schema validation
- Latency and throughput
- Model/version tracking
- Data drift and concept drift
- Prediction quality monitoring
- Security and privacy
- Rollback strategy

14. Practical ML Project Template

Example: CKD risk prediction

Define target and prediction time → clean clinical features → split by patient/time where appropriate → build preprocessing pipeline → establish a simple baseline → compare models → tune on validation/CV → evaluate with clinically meaningful metrics → calibrate probabilities if needed → deploy with strict input validation → monitor drift and performance.

For healthcare applications, ML predictions should support qualified clinical decision-making rather than replace professional diagnosis. Data governance, privacy, validation and clinical evaluation are essential.

15. Interview Revision Sheet

- Bias vs variance?
- Why split train/validation/test?
- What is data leakage?
- Precision vs recall?
- When is F1 useful?
- Why scale features?
- Bagging vs boosting?
- Random forest vs gradient boosting?
- What does PCA do?
- Why use cross-validation?
- L1 vs L2 regularization?
- What is a confusion matrix?

- What is ROC-AUC?
- How do you deploy and monitor a model?

References

- scikit-learn documentation: <https://scikit-learn.org/> — preprocessing, supervised/unsupervised learning, model selection and evaluation. ■cite■turn0search3■
- scikit-learn dataset transformations: https://scikit-learn.org/stable/data_transforms.html ■cite■turn0search14■
- scikit-learn user guide structure: https://scikit-learn.org/stable/user_guide.html — supervised learning, unsupervised learning, model selection/evaluation and common pitfalls. ■cite■turn0search3■